

DockerSSD: Containerized In-Storage Processing and Hardware Acceleration for Computational SSDs

Donghyun Gouk*, Miryeong Kwon*[†], Hanyeoreum Bae*, Myoungsoo Jung*[†]

*Computer Architecture and Memory Systems Laboratory, KAIST

[†]Panmnnesia, Inc.

Abstract—Processing data in storage is an energy-efficient solution to examine massive datasets. However, a general incarnation of such well-known task-offloading model in a real system is unfortunately unsuccessful due to not only poor performance but also many practical challenges, such as limited processing capabilities and high vulnerabilities at the storage-level.

We propose DockerSSD, a fully flexible in-storage processing (ISP) model that can run a variety of applications near flash without their source-level modification. Specifically, it enables lightweight OS-level virtualization in modern SSDs, which allows the storage intelligence to be well harmonized with existing computing environment and makes ISP even faster. Instead of developing a vendor-specific ISP to offload, DockerSSD can reuse existing Docker images, create containers as a self-governing execution object in storage, and process data directly where they are in real-time. To this end, we design a new communication method and virtual firmware that operate together to download Docker images and manage their container execution without a change of the existing storage interface and runtime. We further accelerate ISP and reduce the execution latency by automating container-related network and I/O handling data paths over hardware. Our evaluation shows that DockerSSD is $2.0\times$ faster than state-of-the-art ISP models for workloads with a high volume of system calls or file accesses. Moreover, it demonstrates a reduction in power and energy consumption by $1.6\times$ and $2.3\times$ respectively.

I. INTRODUCTION

In-storage processing (ISP) is a storage model widely used in emerging data analytics. It enables the exploration of large datasets while minimizing data transfer overhead between the host and the storage [1–7]. To achieve this, efforts have been made to incorporate data processing into solid-state drives (SSDs) [8–11]. However, enabling an SSD to cater to the diverse needs of different applications remains challenging.

The main challenge in creating a fully flexible ISP model lies in developing a feasible runtime design and its APIs, rather than the computing support in modern SSDs. Due to the need to protect vendor-sensitive data and intellectual properties, disclosing all internal hardware components and the SSD firmware’s development environment to users is nearly impossible [12]. Therefore, it is the responsibility of SSD vendors to prepare and reveal the ISP runtime and APIs during design. However, constructing an intelligent storage system poses difficulties due to the ever-evolving implementations and requirements of applications. Even with well-structured APIs from SSD vendors, significant source-level modifications are still needed for offloading existing applications. This

also necessitates new host-side daemons to interact with and synchronize the target storage.

The introduction of intelligence at the storage level brings about the problem of managing all I/O requests in block-device format while concurrently handling data near the flash. The simultaneous execution of data processing and block I/O services potentially opens up vulnerabilities and creates problems related to resource protection. As a result, this makes SSDs unreliable for both the ISP tasks and users attempting to process data in storage. For example, due to the SSDs’ lack of understanding about file management and block layout, any users on the host-side can modify the data being processed in storage, which can lead to unpredictable outcomes. Given these challenges, ISP is currently only utilized for specific data-processing applications, such as key-value or object SSDs [13, 14]. Note that, although workloads with a high volume of system calls or file accesses have potential to benefit from ISP, they are also adversely affected by the current limitations of the ISP offloading paradigm. The shortcomings associated with existing ISP models and architectures will be elaborated further in §III.

In this paper, we introduce *DockerSSD*, a fully adaptable ISP model capable of a wide range of data processing close to the flash, requiring no source-level modifications. DockerSSD’s central principle is to implement OS-level virtualization in modern SSDs so that ISP is performed in a containerized manner. This approach ensures seamless integration of storage intelligence with the existing computing environment, speeding decision-making processes. As a result, users do not need to modify algorithms to interact with vendor-specific ISP runtimes, nor require a specialized toolchain to cross-compile the algorithm for the SSD’s firmware.

Though OS-level virtualization could facilitate the wider adoption of ISP across various computing environments, three key challenges emerge when integrating ISP with container methods. First, as containers are self-contained objects that can operate independently in most computing environments, it calls for a unique interface to manage data processing near flash. This interface must ensure compatibility with both the current storage software stack and the network stack. Second, due to the encapsulating nature of containers, which contain their own resources and dependencies, a firmware redesign is necessary to dynamically construct a container and supply an appropriate execution environment. Finally, it is critical that

the processing of container-related requests does not overtax the existing firmware operation or its inherent I/O services.

We introduce an innovative communication approach for in-storage processing (ISP) that leverages Ethernet and collaborates with firmware to autonomously manage the downloading and execution of Docker images. To facilitate containers in storage, we revamp the flash firmware to virtualize the ISP environment and execute data processing without requiring changes at the application level. In addition, we scrutinize the pivotal path of the container-related request service routines and expedite these service processes via specialized hardware. This enhancement makes our virtualization environment more efficient, optimizing it for high-performance ISP.

We summarize the main contributions of this work below.

- **New communication method for ISP.** We present *Ethernet over NVMe* (Ether-oN), a system kernel driver that facilitates network-based ISP management and direct communication between host users and an SSD. Specifically, Ether-oN supports Ethernet networking by allowing asynchronous upcalls and packet overriding within the NVMe standard. This mechanism lets our ISP model utilize a majority of command line interfaces from existing container runtimes, Docker [15]. Ether-oN also allows users to provide data, check ISP statuses, and retrieve results by directly engaging with the SSD, enabling on-demand and real-time data analysis. Since Ether-oN does not necessitate modifications to existing interfaces, it can parallelize network and block I/O services.

- **Containerizing ISP through firmware.** We put forward a novel firmware stack design, called *virtual firmware* (Virtual-FW). Virtual-FW incorporates minimal OS features and a container environment into the SSD by emulating system calls on its bare-metal system. This aims to make system calls lightweight, maintaining ISP system call execution costs on par with function management costs. Virtual-FW can create an ISP task, termed *ISP-container*, from existing Docker images and reliably run it without the overhead introduced by a comprehensive OS. Furthermore, we safeguard SSD resources by employing NVMe namespace and memory isolation, enabling the SSD to process data in a secure, portable sandbox.

- **Hardware accelerated container requests.** We streamline several firmware service routines frequently accessed during ISP-container's data processing. Notably, two critical paths exist in storage-level container runtime: i) the container's network and ii) file I/O services. Network services, bridging communication between a host and ISP-containers, can amplify I/O traffic, hampering performance. On the other hand, File I/O services within ISP-containers require path walking, traversing all root filesystem directories to locate the effective inode and target file, which heightens computational load. To mitigate these choke points in DockerSSD, we engage co-processor accelerators, which handle network packets and execute per-file path walking independently of Virtual-FW.

We construct our firmware and hardware accelerators by carefully examining the crucial kernel components of Linux 4.19 from scratch and validate DockerSSD's functionalities in

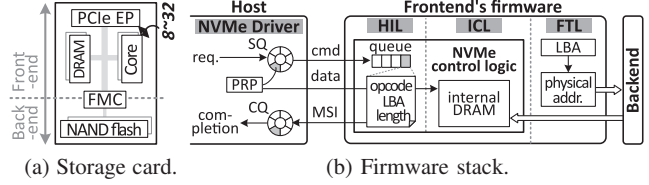


Fig. 1: Architecture of high-performance SSDs.

a real system. For validation, we prototype a high-performance PCIe SSD, incorporating an NVMe controller [16], multi-core processor [17], and hardware accelerators on a 16nm FinFET FPGA chip. DockerSSD successfully runs a range of unaltered applications autonomously. To comprehensively examine the ISP-enabled SSDs design space, we evaluate DockerSSD against various ISP models using a full-system simulation framework [18]. Our assessments reveal that DockerSSD is $1.5\times$ and $2.0\times$ faster than the host system and leading ISP models [8, 9], respectively. From an energy efficiency standpoint, DockerSSD performs $1.6\times$ and $2.3\times$ better than all the tested host and ISP models [8, 9, 19], on average.

II. BACKGROUND

In this section, we briefly explain key architectural modules of an SSD and firmware stack therein. We also describe the necessary components for container and Docker runtimes.

A. High Performance SSDs

Hardware architecture. As shown in Figure 1a, most modern SSDs are primarily structured into two main components: the *frontend* computing complex and the *backend* storage. The frontend incorporates embedded multi-core processor(s) and internal DRAMs, which connect to a PCIe endpoint and the backend via a high-speed interconnect. In contrast, the backend integrates multiple I/O buses, called channels, each linking flash packages through a flash memory controller (FMC). Over recent years, the frontend has greatly improved its hardware capabilities, enabling SSDs to process diverse data in storage. The computational prowess of the embedded processor now rivals that of low-end and mid-range multi-core processors, and the PCIe throughput is increasingly similar to the host-side system bus bandwidth. For instance, embedded processors provide ample computing power to the frontend by demonstrating higher frequencies (e.g., 2GHz 8 cores [13, 20]) and/or incorporating more cores (500MHz 32 cores [21]).

NVMe protocol. SSDs interact with a host by adhering to NVMe, which outlines multiple queues, I/O commands, and data transfer methods over PCIe [22]. Each queue is logically matched with a *submission queue* (SQ) and a *completion queue* (CQ). When a user initiates a request, the NVMe driver submits the request to an SQ and notifies the target SSD of the submission by updating its doorbell register. The NVMe control logic within the storage side can then retrieve the NVMe command from the designated SQ. The command contains memory pointers known as *physical region page* (PRP), indicating the location of data within the host's

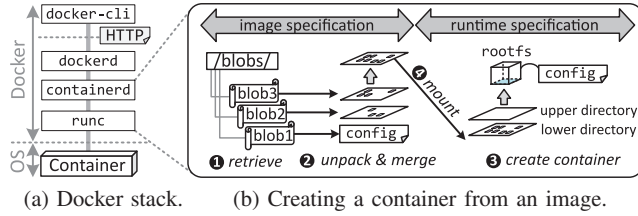


Fig. 2: OS-level virtualization stack.

kernel space. Utilizing PRP, the control logic can actively transfer the data and process the request. Once the I/O service concludes, the control logic finalizes the request by updating the corresponding CQ entry and notifies of this completion through a *message signaled interrupt* (MSI). All these aspects are relevant to the handling of host-side “block” semantic.

Firmware stack. To reconcile the host’s block and flash’s page semantic gap, the frontend employs SSD firmware. As illustrated in Figure 1b, it mainly comprises the *host interface layer* (HIL), *internal cache layer* (ICL), and *flash translation layer* (FTL) [23–25]. HIL/ICL executes the NVMe control logic, aiding NVMe protocol management. Specifically, HIL analyses incoming requests and extracts key I/O information, while ICL relocates data to internal DRAM, managing it as a memory cache. FTL transposes *logical block addresses* (LBAs) into physical flash addresses, dispatches buffered requests, and maintains backend device reliability [26–29]. These components are vital for traditional I/O services, necessitating effective ISP model coordination.

B. OS-Level Virtualization

Container and Docker. Containers leverage cgroups [30] and namespaces [31] to facilitate creation and operation of isolated environments on a control host [15, 32]. These environments effectively act as lightweight virtual machines, excluding preloaded emulation software modules, but including essential applications and runtimes as files [33]. In support of this, the open container initiative [34] provides runtime and image specifications [35, 36], defining container image execution and the manifest that holds meta-information necessary for application launch. Docker extends containers to enable portable deployment, versioning, and component reusability through binary objects or *blobs* [37]. A blob encapsulates bundled applications, allowing for transparent transfer and quick installation on “any” Docker-enabled system.

Virtualization services. Figure 2a depicts the Docker stack we use. Users interact with the Docker daemon (*dockerd*) via command-line interface (*docker-cli*). *dockerd* enables Docker-specific features like container volume management and Ethernet communication via HTTP REST API [38, 39], and sets up a virtual network for communication between users and containers. Underneath, *containerd* handles the image and its container. The low-level runtime (*runc*) configures namespaces and cgroups, and it initiates the target container.

Figure 2b details the container creation from an image. First, Docker retrieves a blob from a user-defined location

(1). *containerd* then unpacks the blob following image specifications (2), yielding a configuration file and image layers. *containerd* merges the layers to create a read-only *lower directory*, with help from a Docker storage driver (i.e., overlay). Lastly, *runc* forms a writable *upper directory* and merges it with the lower directory to get a root filesystem (*rootfs*) (3). Note that *rootfs*, with all runtime configurations, allows independent operation of the target application in varied execution environments (4).

Note that *cgroups* and *namespaces* are OS kernel features, not the functionalities of containers. Even though containers leverage the two kernel building blocks, the execution environment is all defined by runtime and image specification. The cgroups and namespaces in OS-level virtualization are only used for multi-tenant support by throttling requests and isolating resources, respectively. Instead, Docker stack is the key to mount and execute containers since each created container itself is just a process on a Docker-enable system.

III. CHALLENGE CLASSIFICATION AND ANALYSIS

For widespread acceptance of ISP, the optimal ISP model needs to be user-friendly and align seamlessly with the present computing environment, including architectural interfaces and software infrastructures. However, most ISP models grant users the ability to offload one or more specific portions of the intended applications to the underlying storage using a pre-determined set of static kernels or by reprogramming them with firmware-level APIs [8–11, 40–47]. Because these static kernels and APIs have to be implemented suitably during the device design phase, the computational capability of the ISP is confined in various aspects. Specifically, the existing ISP models encounter five key challenges: i) *manual ISP implementation*, ii) *disregard for file layout*, iii) *kernel context switching*, iv) *device reliance*, and v) *data vulnerability*.

In this section, we expound each of these challenges and scrutinize their performance implications, making the optimal, flexible ISP model challenging to implement in a real-world system. We will also elaborate on how these challenges limit the previous works in §VIII.

A. Fully Flexible ISP and Challenges

Manual ISP implementation. Given a set of static kernel libraries or APIs, users are required to manually identify the sections of the application that can be assigned to the underlying SSD. During this process, users need to be aware of the precise location of a reference runtime (or static kernel library), and they must segregate the code segments for offloading from the original application. Existing ISP models then necessitate the replacement of these code segments with a suitable static kernel library or the composition of the kernel using the provided APIs, all manually executed.

Disregard for file layout. Most applications manage storage-related data via a filesystem, but firmware lacks knowledge of its file data layouts. Hence, users must obtain a set of LBAs linked with the file offset, then convey this LBA set to the offloaded ISP kernel. Consider an example of kernel offloading

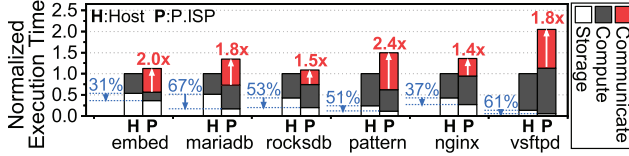


Fig. 3: Performance impact analysis.

in a deep-learning recommendation model system (DLRM [1]), increasingly accelerated by near-data processing [48–52]. ISP can consolidate DLRM’s feature embeddings, returning only a subset of results, thereby addressing data movement overhead between host and storage. However, ISP needs to locate sparse features stored in backend flash as embedding files, but lacks knowledge of their locations. Note, data from each file is distributed across various block locations, managed by the host-side filesystem, not firmware.

Kernel context switches. As the current ISP models cannot accommodate an application in a full-fledged manner, it requires switching contexts of different kernels at the host-level. These context switches accompany frequent host interventions and introduce a new type of data transfers. For instance, DLRM’s ISP kernels require different input contexts such as operation types (e.g., sum, average), and embedding table indices for appropriate embedding reduction. During context deliveries between host and SSD, such information should also be appropriately serialized and deserialized, which increases data transferring overhead more.

Device reliance. Users must recompile both the target application and each kernel to offload in diverse ways. In contrast to the application, these kernels must run as a component of the base firmware in most prior ISP models, necessitating cross-compilation using a suitable development toolchain advised by the SSD’s vendor. This cross-compilation introduces a device dependence problem. Even with successful manual ISP implementation, all existing ISP models face challenges of reduced portability and compatibility across different firmware versions and storage configurations.

Data vulnerability. Despite overcoming the previously mentioned challenges, existing ISP models tend to neglect vulnerability issues caused by concurrent execution between the host and storage. It is important to note that SSDs function as a “block” device, shared by all host applications via a filesystem. As there are no mechanisms for resource and data protection on the storage side, both the host application and ISP kernels can access the same flash location simultaneously. Regrettably, this can create unpredictable scenarios and raise significant vulnerability concerns in existing ISP models.

Note that current ISP models do not exhibit all the five challenges at the same time. The presence of these challenges varies depending on the system’s circumstances and execution environment. Even though ISP models could gain advantages from workloads involving extensive system calls and/or file I/O operations, their performance may be substantially impaired due to the prevalent ISP offload approach. We further delve into this discussion in §VIII.

B. Performance Impact Assessment

We conduct an analysis on a revised iteration of the Programmable-ISP model [8, 9] (referred to as P. ISP), assessing a broad range of benchmarks that predominantly engage in system calls or file accesses. Detailed characteristics of these workloads and the evaluation environments are delineated in §VII. Figure 3 presents the results, breaking down the ISP execution times into ISP computation latency (Compute), storage backend’s delay (Storage), and latency for communication and synchronization between the host and ISP storage (Communicate). To aid comprehension, we juxtapose the Programmable-ISP with a host-only system (Host).

Despite Host demonstrating performance variations across workloads, substantial potential exists to cut down application latency, given that Storage comprises 38% of the total execution time on average. P. ISP, by processing data in storage, mitigates the data movement overhead between host and storage, leading to a 50% average reduction in Storage compared to Host. Regrettably, the overall end-to-end latency of P. ISP grows by 1.4×, rendering ISP somewhat less appealing. The primary culprit for this performance decline is Communicate, accounting for 43% of the P. ISP latency.

As depicted in the figure, rocksdb has the least overhead for Communicate. This is attributable to its context switches having fewer parameters (e.g., database file numbers) to handle, despite its kernel execution time being longer than others (e.g., compaction [53]). In contrast, pattern escalates Communicate by 2.4× on average. As pattern returns multiple lines that match an input query word, the per-kernel data exchange size is more significant than others, varying from tens of KB to tens of MB. Compute may sometimes be higher or lower, resulting from the SSD’s lower computing power compared to the host but lacking the burdensome OS involvement in accessing backend media. We will delve into these performance trends in §VII in detail.

Note that if we can render ISP models host-independent and manage their execution autonomously, we can eliminate communication overhead, enhancing performance with lower computing power in many instances. Although complex to quantitatively analyze overheads of manual ISP implementation, device reliance, and data vulnerability, a virtualized ISP model can tackle these as a portable sandbox.

IV. CROSS-LAYER OPTIMIZED ISP CONTAINERIZATION

Instead of executing a series of static kernels or tasks, DockerSSD creates a virtual ISP environment. This allows the host to process data near flash in containerized applications, termed *ISP-containers*. In this section, we discuss achieving ISP containerization via firmware, which addresses challenges of manual ISP implementations and context switches. We demonstrate how DockerSSD manages backend media to alleviate simultaneous access, termed as data vulnerability. Containers are self-contained objects running independently on

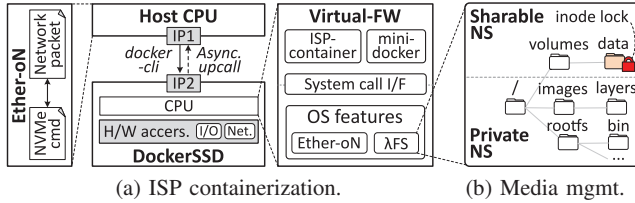


Fig. 4: High-level view of DockerSSD.

various systems; our firmware-level virtualization can eliminate device reliance and file layout issues.

A. High-level Overview of DockerSSD

To implement containerization in storage, DockerSSD addresses three technical hurdles. First, a new type of communication interface compatible with both existing storage and network stack is required for ISP-container management. Second, the underlying firmware must recognize the structure of containers and support their execution with minimal overhead. Lastly, containers operating within the storage should not disrupt block I/O services or degrade performance.

Figure 4a provides an overview of DockerSSD. To tackle the aforementioned technical challenges, it incorporates three key software and hardware components: i) a system driver that introduces a novel ISP management interface (*Ether-oN*), ii) a firmware environment equipped with Docker stack (*Virtual-FW*), and iii) hardware accelerators to expedite container-related I/O services. Generally speaking, *Ether-oN* enables socket-based Ethernet communication over PCIe, overriding NVMe. This virtualization is achieved by its driver supporting network-to-storage packet translation and an asynchronous upcall mechanism. Thanks to *Ether-oN*'s ability to assign individual IP addresses to each endpoint, users can issue ISP-related requests to *Virtual-FW* via *docker-cli* at any time. The specifics of *Ether-oN* will be elaborated in §V-A. Meanwhile, *Virtual-FW* introduces minimal OS features into the SSD, emulating the OS system call interfaces on the bare-metal SSD hardware. It also utilizes *mini-docker*, which implements the core functions of the Docker stack, thereby enabling firmware-level ISP containerization. More details about *Virtual-FW* are provided in §V-B.

In this study, we demonstrate that the collaboration between *Virtual-FW* and *Ether-oN* enables users to operate full-scale applications close to flash. This is achieved using familiar Docker commands, eliminating the need for vendor-specific interfaces. Figure 5 illustrates this process. Users initiate a blob download from *Virtual-FW* via *docker pull* (1, 2) and then launch the associated ISP-container with *docker run* (3). This process does not necessitate any changes at the application level (4). During the operation of the ISP-container, users have the ability to submit non-I/O, ISP-centric requests via *Ether-oN* (4). For instance, users can send a query using *mariadb-cli*, which handles the request over Ethernet. Although *Virtual-FW*'s system call emulation (and the implementation of firmware-level features/functions) is engineered to keep the ISP-container execution cost low, the

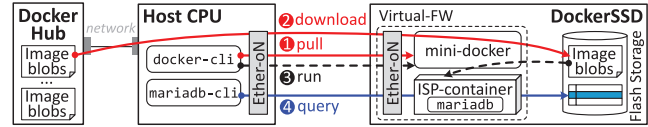


Fig. 5: Application execution example.

ISP-container itself can produce I/O and network requests for data processing and communication. As shown in Figure 4a, DockerSSD employs two hardware accelerators, each designed to minimize overhead caused by frequent container-related requests utilizing *Virtual-FW*'s I/O and network features. These accelerators will be further detailed in §VI.

B. Backend Media Management

DockerSSD introduces the *Lambda filesystem* (λ FS) for effective backend flash management, as depicted in Figure 4b. For seamless sharing of files and directories between the host and storage, λ FS integrates the file structures employed by the host, which is founded on EXT4. This ensures protection against any ISP-container that might be unauthorized to access these files as well. To effectively handle file concurrency and safeguard against vulnerabilities, λ FS segments the media into two separate NVMe namespaces with the support of the NVMe subsystem [22]: i) the private namespace (*private-NS*) and ii) the sharable namespace (*sharable-NS*). The private-NS remains hidden from the host while the sharable-NS is accessible to both the host and ISP-container. In this setup, our NVMe subsystem (managed by HIL) exposes two PCIe functions (akin to a storage-side port), each associated with *Virtual-FW* and the host, respectively. The namespace list of the former function includes both the private- and sharable-NS, while the latter contains only the sharable-NS.

λ FS allocates the private-NS for container and OS-level virtualization runtimes. Thus, ISP-related contents needing protection, such as image layers (*/images/*) and container data (*/rootfs/*), are kept invisible to the users. Conversely, the sharable-NS is used for data that the host needs to place/retrieve and ISP-container processes in storage. This setup allows ISP-containers and users to share necessary input/output data, but it necessitates a lock for the data across all I/O accesses. To tackle this, λ FS employs a new in-memory data structure, the *inode lock*. This lock synchronizes with the host's *inode* cache via *Ether-oN* when an ISP-container binds a host FS's directory or file to λ FS for its data processing. For this synchronization, λ FS adds a reference counter to the *inode* of *virtual file system* (VFS) on the host, protecting it with the existing *inode*'s semaphore. This counter is updated when the target file (including its directory file) is opened or closed. VFS and λ FS then send a special packet via *Ether-oN* to update it. The file is thus accessible only if the *inode* reference counter is zero, resolving the concurrent access issues. When the underlying ISP-container gets the file access permission, VFS invalidates its *inode* cache, enabling it to refer to the storage's latest information.

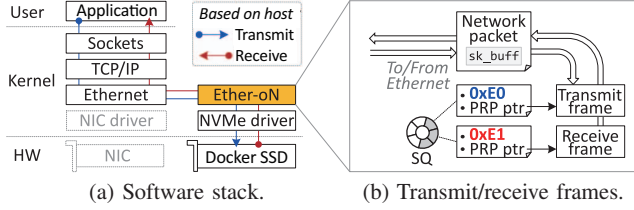


Fig. 6: Interface design and architecture (Ether-oN).

Take note, as two different systems can concurrently access the target partition, it is possible for the host to have a stale *inode* within the VFS. This could potentially corrupt the filesystem and/or make files or directories, updated by the ISP-container, inaccessible. Thus, if DockerSSD binds to a portion of the host-side FS, it must ensure synchronization between the host and ISP-container, even in the absence of host participation. Furthermore, this locking mechanism aims solely for synchronization between the host and ISP-container; its persistence is unnecessary. Even during power failures, the lock does not have to remain since the host can restore the file system and restart the container from scratch.

V. INTERFACE DESIGN AND FIRMWARE VIRTUALIZATION

A. Ether-oN: Ethernet over NVMe

The Docker stack and container interface typically utilize Ethernet by default, and therefore, we have chosen Ethernet for ISP-related service communication by overlaying standard socket-based networking on the NVMe standard, termed Ether-oN. The architecture of Ether-oN is depicted in Figure 6a. Host applications, which need to communicate with DockerSSD, operate on a conventional network stack and interact with Virtual-FW via Ethernet. Ether-oN facilitates communication between applications on the host and the underlying ISP-container using Ethernet, such as TCP/IP. This is achieved by establishing an intranet between the host and DockerSSD, which does not require source-level adjustments for the communication. To this end, Ether-oN's kernel driver establishes a network adapter on the host that links directly to DockerSSD. This adapter integrates with the host's network stack, enabling it to transform Ethernet packets from the network stack into NVMe commands, which are then dispatched to the designated DockerSSD.

B. Virtual-FW: Docker-Enabled Firmware

While Ether-oN enables network-based communication on block devices, existing firmware lacks flexible computing and resource management for Ethernet in ISP. To rectify, Virtual-FW incorporates essential OS features and a container environment into current I/O service path (HIL $\Rightarrow$ ICL $\Rightarrow$ FTL). **Integration of OS Features.** Figure 7a demonstrates how our ISP service path is integrated into the existing I/O service path at the firmware-level. DockerSSD implements three handlers that each manifest OS features related to thread, I/O, and network management. These handlers are positioned between

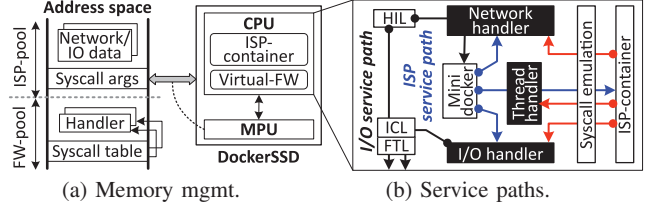


Fig. 7: Internal organization of our Virtual-FW.

HIL and ICL. As illustrated in Figure 7b, the *thread handler* manages DockerSSD's bare-metal DRAMs in a page-granular manner, but partitions them into two memory regions to protect Virtual-FW: the *FW-pool* and the *ISP-pool*. The FW-pool houses a table that implements all three handlers through system call emulation interfaces, whereas the ISP-pool manages call arguments and all other data types. The execution of ISP-containers and Virtual-FW is distinguished using DockerSSD's CPU mode, and its memory protection unit (MPU) only permits execution with privilege mode to access the FW-pool, thereby safeguarding Virtual-FW from ISP-container executions. Note that since Virtual-FW can also access the ISP-pool (with the privilege mode), there is no need for data copying between the FW-pool and ISP-pool, thereby eliminating the data overhead caused by mode switching.

The *I/O handler* manages only the I/O requests that ISP-containers generate for their data processing in storage. Thus, it does not need to have heavy block management components such as the block layer and NVMe software stack. The I/O handler implements λ FS (§IV-B) and exposes them through Virtual-FW's emulation interfaces. Specifically, it traverses the LBA set associated with a given file name, called *path walking* operations, and caches the set for fast accesses, referred to as *I/O node caching*. Since these operations are often requested from an ISP-container running in the SSD, we also accelerate them through hardware as an advanced technique of Virtual-FW (§VI). Since all network requests should go through the host's network stack and are handed over to Virtual-FW, the network handler does not require security features such as IP security and Netfilter. The network handler implements TCP/IP features associated with communication channel establishment and network packet management. Specifically, the network handler employs a TCP finite state machine (FSM) managing states of socket communication. It also creates and parses network packets for the communication channel through packet encapsulation [54].

System call emulation. Virtual-FW also emulates system calls to facilitate seamless container execution in storage, without requiring any application-level changes or recompilation. Given the SSD's limited computing capabilities, our system call emulation is implemented through a lightweight function wrapper. Table Ia provides an overview of the types of system calls emulated, the quantity, and which handler manages which call. This emulation substantially reduces memory overhead by eliminating all overriding system functions from the call path. For instance, *glibc's open* internally calls *openat*, which

Handler (# syscall)	Category	Example	Category	CLI command
Thread Handler (65)	Process management	fork exit	Image management	docker rmi docker pull
	Memory management	brk mmap	Container life cycle management	docker create docker run
	Inter-Process comm.	pipe mq_open		docker start docker stop
I/O Handler (43)	Lock & signal mgmt.	futex		docker restart docker kill
	File/dir mgmt.	openat mkdir		docker rm
	File I/O & link	read symlink	Container monitoring	docker logs docker ps
Network Handler (25)	Permission	chmod chown		
	Polling APIs	epoll_create		
	Socket APIs	socket bind		
	Network comm.	sendto		

(a) System calls.

(b) Docker commands.

TABLE I: Virtual-FW's OS features and supported commands.

is primarily designed to support different CPU architectures and ensure compatibility but isn't necessary for Virtual-FW.

From a call latency perspective, our system call emulation can decrease the context switch overhead, thereby minimizing the strain of running containers on an SSD. A fully-fledged OS typically performs context switches when system calls return to userland, but our emulation system does not have such a boundary due to its function-level implementation.

However, this brings up two technical challenges. Firstly, unlike existing network environments, NVMe lacks a mechanism to handle “in-bound” requests. Even though there are not any PCIe devices capable of issuing an NVMe command to the host, ISP-containers require such an upcall mechanism for communication with users. Secondly, host users need to access the SSD via both the network and storage stacks for their ISP services and block I/O services. To resolve these challenges, we introduce two NVMe vendor-specific commands, which we will discuss further.

Network support using NVMe. The two new NVMe commands introduced in this work are i) *transmit* and ii) *receive* frames. The structure of these frames is exactly the same as that of NVMe commands, but they use different operation codes (0xE0~0xE1), which are reserved for vendor-specific in NVMe (cf. Figure 6b). With the NVMe frames, DockerSSD can manage Ethernet over NVMe. Specifically, when our Ether-oN driver receives an Ethernet request, it extracts the network packet buffer (`struct sk_buff`) from the frame and allocates a kernel page (4KB-aligned). It copies the `sk_buff` to the kernel page, including network headers, payloads, and checksums. The Ether-oN driver then creates an NVMe command, filling its PRP field with the system address of the kernel page. It also sets the operation code as a transmit frame and submits it to the underlying NVMe driver.

Enabling inbound network services. To enable SSD to request service to the host (such as network response, remote accesses), we introduce an asynchronous upcall mechanism leveraging pre-allocated NVMe commands. At kernel initialization, Ether-oN sends a set of NVMe commands to SSD via NVMe SQ, each representing a receive frame, even without an Ethernet request. For each NVMe command, Ether-oN prepares a kernel page and inserts a reception code. At this stage, DockerSSD continues to submit NVMe commands without action for I/O completion. When an ISP-container

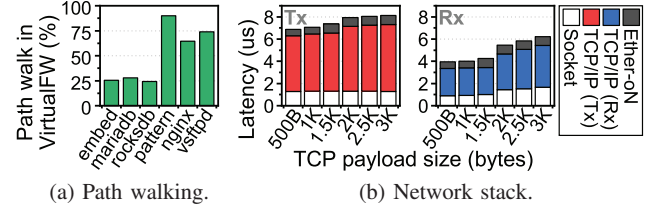


Fig. 8: Overhead analysis in file and network management.

sends an Ethernet frame back to the host, DockerSSD copies its `sk_buff` to the pre-allocated kernel page and completes the outstanding command. Ether-oN converts the completed command into an Ethernet frame and returns it to the network stack. For future communication, it sends another receive frame to DockerSSD. Note NVMe can issue as many upcall requests as SQ entries. Our preliminary studies led us to use four pre-allocated commands per SQ.

Firmware-level container environment. The container environment within The container environment within DockerSSD aims to store, run, and manage containers near flash memory, rather than enabling full Docker functionalities in an SSD. To achieve this, Virtual-FW introduces mini-docker that implements the necessary 10 Docker commands (out of 106 commands [39]), summarized in Table Ib. Specifically, mini-docker establishes a container environment linked with `dockerd`, `containerd`, and `runc`.

Similar to existing `dockerd`, mini-docker communicates with the host's `docker-cli` via HTTP. To handle an ISP-container, a user simply issues an HTTP command, including DockerSSD's IP address and the image name to be executed. Mini-docker in Virtual-FW parses the Docker command and performs the appropriate action. For instance, if an image management command (`docker rmi/pull`) is issued by the host, mini-docker stores the corresponding image blob and manifest via λ FS (at `/images/blobs` and `/images/manifest` respectively).

Note that the manifest file contains information about the target application (such as the `entryscript`) and all the image layers needed for its `rootfs`. When the host issues commands to create or start a container (`docker create/start`), mini-docker calls the thread handler and generates an ISP-container, referring to the `entryscript` (similar to `containerd`). It then mounts the `rootfs` to the ISP-container. To monitor the ISP-container while running, mini-docker stores logging information like `stdout` and `stderr` in λ FS (at `/containers/<id>/rootfs/log`). These logs can be transferred to the host through Virtual-FW's network handler, making them available for real-time analysis.

VI. CONTAINER SERVICE HARDWARE ACCELERATIONS

A. Performance Bottleneck Analysis for ISP Computing Path

Since our ISP-container model can operate autonomously, it introduces a new type of in-storage computing activities that which may require acceleration over hardware.

ISP-container file accesses. Figure 8a illustrates the fraction of time the path walking process consumes in the execution of

Virtual-FW. Path walking is frequently invoked when an ISP-container needs to access underlying files using `open()` and `stat()` functions. In this performance bottleneck analysis, we use the same set of workloads and testing environment as in the previous section (cf. §III-B). Data-intensive workloads such as `embed`, `mariadb`, and `rocksdb` typically open a large file and continue operations (e.g., queries) within that file. Consequently, path walking accounts for only 25.8% of the Virtual-FW execution time. However, when analyzing service-centric workloads (e.g., `pattern`, `nginx`, and `vsftpd`), path walking occupies a significantly larger fraction of the ISP-container execution time, averaging 76.1% of the total latency. These workloads access the flash media as frequently as the data-intensive workloads. However, their per-file services and operations are generally faster than file accesses, which places path walking on the critical path.

Firmware-level network services. On the other hand, Figure 8b breaks down the latency of the transmission (Tx) and reception (Rx) network stack of Virtual-FW into the time consumed by Ether-oN, TCP/IP, and socket management. In this analysis, we utilize a network microbenchmark [55] to send and receive a variety of network packet sizes (within the storage), ranging from 0.5KB to 3KB. As can be observed from the figure, TCP/IP is the primary contributor to network latency, surpassing Ether-oN and socket management. Specifically, TCP/IP accounts for 73.2% and 59.9% of the Tx and Rx network latency, respectively. This is due to the need for TCP/IP to manage all packet header and payload information, along with a connection tracking mechanism and state/congestion control. Network communication is conducted asynchronously, which means this latency can overlap with ISP computing time. However, delays can occur in ISP-container execution when the ISP-container is waiting for a response, or when there is an excessive number of packet requests.

B. Specifics of In-Storage I/O and Network Accelerators

We address the overhead imposed by the path walking and network service mentioned above using two domain-specific accelerators: i) *In-storage I/O Accelerator* (IIO-X) and ii) *Network Accelerator* (NET-X). IIO-X takes the path walking off the critical path on file accesses, whereas NET-X shortens TCP/IP times by automating the packetization over hardware. **Accelerating path walking.** DockerSSD works to minimize the latency overhead attributed to the path walking process. When an ISP-container opens a file, Virtual-FW informs IIO-X of the corresponding file path by passing it through MMIO. IIO-X then retrieves the `inode` located on the backend flash by simultaneously searching the container's lower/upper directories, thus further reducing the I/O access latency. Afterwards, IIO-X caches `inode` in internal SRAM to decrease path walkings for ISP. This cached `inode` accelerates path access times as it also conserves special-purpose `inodes`, directing to root (/) or volume mounts. Coherence between host and storage remains intact, as `inode` lock in λ FS nullifies `inode` cache.

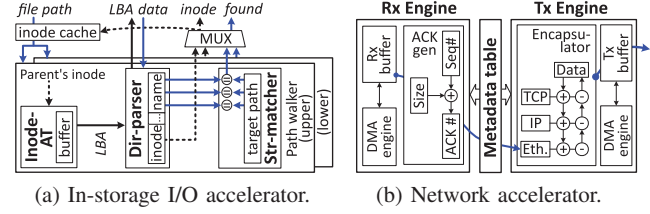


Fig. 9: Internal architecture of our hardware accelerators.

Figure 9a provides a detailed look at DockerSSD's IIO-X design and implementation. The IIO-X employs two path walker units, each responsible for searching the upper and lower directories. For instance, when an ISP-container opens the file `/a/b.txt`, IIO-X performs parallel path walking for `/upper/a/b.txt` and `/lower/a/b.txt`. To accomplish this, each hardware logic in IIO-X initiates from the root directory `/` and continuously iterates the path walking process until the target file is located. This automated operation involves three core hardware modules: i) *inode address translator* (inode-AT), ii) *directory parser* (dir-parser), and iii) *string matcher* (str-matcher). The inode-AT module in IIO-X calculates the LBA of a given `inode` by referencing λ FS's metadata, such as the `inode` table. This module collaborates with the ICL and FTL to load the `inode`'s data into the inode-AT's buffer (256B). Since this data is exclusive to Virtual-FW, the ICL can bypass and avoid caching it, thus mitigating data copy overhead between ICL and IIO-X.

Following this, the dir-parser checks the `inode`, retrieves LBAs it refers to, and, with aid of ICL/FTL, fetches the associated directory data. It then parses child file names. The str-matcher subsequently compares these parsed names (children) with a directory or file name in upper/lower directories. The str-matcher's design is based on string pattern matching hardware inspired by [56], incorporating a set of character comparators and shift registers to compare individual characters and track their sequence for pattern matching. If the target path contains more subdirectories, the str-matcher sends the relevant `inode` to inode-AT. Once the final target is located by IIO-X, it returns the file `inode` of the target path. If the search fails, it informs Virtual-FW accordingly. In scenarios where both path walker units in IIO-X locate the target, the upper directory is given priority, and its `inode` is returned (without any software intervention).

Automating TCP/IP in storage. As illustrated in Figure 9b, NET-X comprises three hardware modules: i) receiver (Rx) engine, ii) transmitter (Tx) engine, and iii) metadata table module. The primary task of the Rx engine is to manage acknowledges (ACK) for the inbound packets requested by the host or loopback requests originating from the mini-docker/ISP-container. Since DockerSSD connects to the host via PCIe (not a network), we can design the Tx/Rx engines without needing data integrity verification mechanisms like checksum. As a result, the Rx engine can generate the ACK number for an incoming packet by merely combining the TCP's sequence number with the packet size.

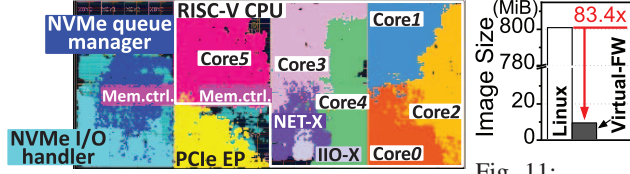


Fig. 10: DockerSSD implementation.

Fig. 11: Image size.

Conversely, the Tx engine is tasked with generating suitable packets for outbound requests from ISP-container or mini-docker, or ACKs received from the host. It formulates packets using its encapsulator hardware logic, which understands characteristics of network headers. The encapsulator utilizes destination and source ports/addresses from the TCP, IP, and Ethernet headers to generate packets – information initially stored by Rx engine. Despite absence of a checksum, the TCP header’s sequence number needs to be filled. Therefore, the Tx engine computes total bytes it generates and uses this figure for TCP sequence number generation.

Importantly, the Tx engine handles all outbound packets, and it needs to be aware of the pre-allocated NVMe commands by Ether-oN. The PRP data, stored in the Tx buffer by Ether-oN, allows NET-X to manage Tx packets through NVMe. Most of the information needed by the Tx/Rx engines, such as TCP’s source/destination port numbers and IP/MAC addresses, is stored in the metadata table module. This module is designed as a dual-port memory, enabling simultaneous access.

VII. EVALUATION

Prototype and methodology. We constructed a DockerSSD hardware prototype on a 16nm FinFET FPGA chip [57]. To validate in a real system, we adapted NVMe hardware IPs [16] to incorporate a multi-core processor [17] and integrated RTL into DockerSSD’s frontend. As depicted in Figure 10, the prototype features a queue manager, an I/O engine, and a PCIe controller managing all hardware-related NVMe tasks. It connects six RISC-V in-order cores running our Virtual-FW via AXI/TileLink buses [58, 59]. We used two DDR4 controllers to emulate flash for the backend, modifying a multi-channel flash timing emulation model [60]. We made modifications to 3.4K LOC for Ether-oN/ λ FS and altered 7.5K LOC for firmware and storage-level modules based on EXT4. We tested these functionalities on our prototype; Figure 11 compares the actual image size of Linux and Virtual-FW. The target binary was reduced by a factor of 83.4 $\times$, making Virtual-FW compatible with the embedded processor.

However, it is virtually impossible for a single device to cater to diverse hardware/software configurations and varied ISP communication methods across a spectrum of ISP models. To explore the full scope of design and performance spaces, we assessed various ISP models on a full system simulator, gem5 [61]. Our simulation environment, cross-validated with the prototype’s backend synthesis, integrates a cycle-accurate SSD simulator [18] and power model [62]. The host uses a 3.8GHz CPU and 64GB DDR4, while storage is set as an NVMe SSD

Program	Workload	I/O size	I/O count	# sys-calls	# path-walk	# files-opened	# TCP-packet	Exec. Time
embed-[1]	rm1 [69]	1.3GB	317K	1.3M	9K	260	0	8s
	rm2 [49]	5.8GB	1.4M	1.7M	9K	320	0	24s
mariadb-[2]	tpch4 [70]	17.1GB	1.1M	1.1M	37K	250	160	25s
	tpch11 [70]	6.2GB	400K	361K	38K	260	190	8s
rocksdb-[3]	read [71]	4.1GB	431K	1.1M	9K	1.2K	0	14s
	write [71]	18.5GB	24K	285K	9K	3.6K	0	24s
pattern-[4, 5]	find [72]	2.4GB	381K	1.8M	359K	352K	0	11s
	line [72]	1.7GB	262K	1.7M	476K	235K	0	11s
	word [72]	2.1GB	340K	2.2M	618K	307K	0	10s
nginx-[6]	web0 [73]	7.5GB	126K	665K	126K	4.4K	543M	9s
	web1 [73]	0.9GB	50K	344K	109K	2K	154K	3s
	filedown [74]	13.5GB	109K	30K	1K	40	155K	6s
vsftpd-[7]	fileup [75]	12.1GB	93K	5.4M	127K	115K	1.2M	2s

TABLE II: Workload characteristics.

[63], with 48 MLC flashes across 12 channels. The frontend has a 2.2GHz processor and 2GB DRAM, configurations similar to those in previous studies [10, 13, 64].

Data processing models. We have created seven unique data processing models for our study. The *Host* model, representing the above-mentioned host, serves as our baseline non-ISP model. We then implemented two programmable ISP models, *P.ISP-R* and *P.ISP-V*, by utilizing the offloading model and interface from the previous studies [8, 9]. Specifically, *P.ISP-R* uses gRPC [65] as its programmable interface [8], while *P.ISP-V* employs NVMe vendor-specific commands to minimize interface overhead [9]. Both *P.ISP-R/V* are designed with the presumption that they encompass the majority of ISP functions users intend to delegate, excluding system-specific modules like file I/O management. Therefore, *P.ISP-R/V* uses the host’s runtime and file systems when necessary. Note that, in our setup, *P.ISP-R/V* only utilizes an LBA set when the ISP kernels access a new file, thus reducing overhead. We also implemented four DockerSSD variations. The *D-Naive* model utilizes separate processor and controller complexes, each running an ISP-container and firmware with fully-fledged Linux [13, 19, 66–68]. *D-FullOS* modifies the *D-Naive*’s hardware architecture with DockerSSD’s, enabling co-running of the ISP-container and firmware on the same computing complex. *D-VirtFW* employs Virtual-FW on *D-FullOS* hardware as a replacement for the full-fledged OS. Lastly, *D-CoDesign* optimizes path walking and network service (used in *D-VirtFW*) over two hardware accelerators, *IIO-X* and *NET-X*.

Benchmarks and workloads. We have chosen 6 benchmarks, encompassing 13 unique workloads. These include: i) DLRM’s embedding operations (‘embed’ [1, 76]), ii) relational database (‘mariadb’ [2]), iii) key-value store (‘rocksdb’ [3]), iv) text mining (‘pattern’ using [4, 5]), v) web server (‘nginx’ [6]), and vi) file server (‘vsftpd’ [7]). Specifically, embed performs an embedding table lookup and aggregates/reduces a given set of sparse features. rm1 uses features longer than rm2, but the number of table entries is half of that of rm2. mariadb uses TPC-H [70] representing business database workloads. rocksdb read/write performs Get/Put queries on more than 100K keys. The pattern workload has three variants: ‘find’,

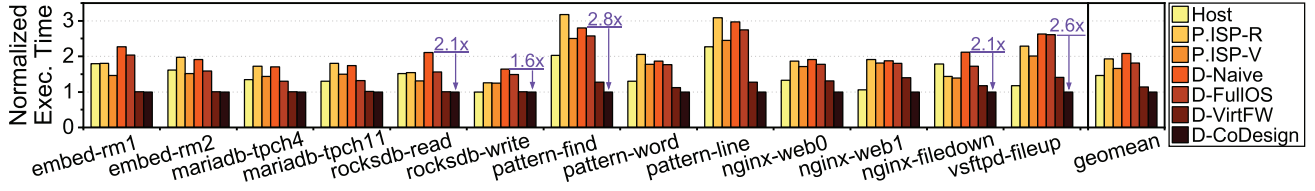


Fig. 12: Overall performance analysis.

‘line’, and ‘word’, each of which searches a text and counts the number of lines/words in more than 20K documents. The nginx workload comes in two variations: ‘web0/1’, which serves static webpages, and ‘filedown’, designed for video file streaming. vsftpd’s ‘fileup’ is designed to upload thousands of image files per service. Each workload was run 10 times to provide a robust evaluation. To be precise, the specific workloads tested for each benchmark are outlined in Table II. The table also analyzes the number of I/O requests, size of processing data, number of system calls, path walk operations, and TCP packets generated when each application runs.

A. Overall Performance Comparisons

Figure 12 compares the execution time of various ISP models, normalized to the latency of Host. To be accurate, we also provide the actual latency for Host in Table II. For conciseness, we use seven abbreviated terms as follows: i) Network operation times (*Network*), ii) Kernel’s context switches (*Kernel-ctx*), iii) Handshaking of the set of LBAs (*LBA-set*), iv) SSD accesses (*Storage*), v) System call and OS stack latency (*System*), and vi) ISP kernel latency (*Compute*). **Programmable-ISP.** While P.ISP-R/V can eliminate data transfers associated with data processing, their overall performance is inferior to Host. This performance drop is mainly due to Kernel-ctx and LBA-set, as discussed in §VII-B. P.ISP-R/V outperforms Host only in rocksdb-read and nginx-filedown scenarios. Due to Get operations in rocksdb/nginx, system and data movement significantly influence their end-to-end latency. These OS and system call-related burdens are not present in P.ISP-R/V, as their ISP kernels operate on the device’s bare-metal system, delivering better performance than Host. We will delve deeper into this in §VII-B. Note that P.ISP-V exhibits 14.5% less latency than P.ISP-R, as P.ISP-V does not handle the RPC protocol of P.ISP-R and requires no network responses.

ISP-container with OS. Although D-Naive/Fullos eliminate the communication overhead (Kernel-ctx+LBA-set), they unfortunately grapple with internal computational burdens and data movement issues stemming from low power hardware and a complete software stack. The time D-Fullos spends

handling system calls and the OS for ISP services with its low-power computing makes it 9.3% slower than P.ISP-V. Besides this software overhead, D-Naive requires transferring data between the computing processor (designated for running ISP-containers) and firmware controller complexes, resulting in an average slowdown of 11.8% compared to D-Fullos.

FW-level containerization (DockerSSD). D-VirtFW leverages the benefits of running applications in a full-fledged manner while eliminating the software stack overhead of D-Fullos and hardware burden of D-Naive. It outperforms P.ISP-R/V, D-Naive, and D-Fullos by 1.6×, 1.9×, and 1.6×, on average, respectively. Our ISP-container accesses the backend flash via λFS, requiring no LBA-set, resulting in an average latency reduction of 9.5% compared to P.ISP-R/V. Furthermore, all execution parameters are pre-packaged in rootfs, eliminating Kernel-ctx and improving ISP performance by 30.9% compared to P.ISP-R/V. We will delve into each component’s benefit and latency reduction (including OS-feature/System) in §VII-B.

D-CoDesign delivers ISP performance similar to D-VirtFW for workloads processing large amounts of data with few file accesses (e.g., embed, mariadb, and rocksdb). However, it shows 22.3%, 29.8%, and 40.6% better performance than D-VirtFW for pattern, nginx, and vsftpd, on average, respectively. Given that their ISP-containers need to process data across multiple files or network accesses for media services (cf. Table II), IIO-X and NET-X reduce path walking and network latency by 8.7% and 3.7%, on average, respectively. As a result, D-CoDesign has a 34% shorter execution time than Host across all tested workloads.

B. In-Depth Examination of Latency Reduction

Figure 13 dissects the end-to-end latency into the 7 latency components defined in §VII-A. In this subsection, we explore varying behaviors of ISP models and select rocksdb-read, rocksdb-write, pattern-find, nginx-filedown, and vsftpd-fileup as representative workloads for better understanding.

Computation and data movement. Three main observations can be made from this figure. Firstly, the Compute component of all ISP models is lengthier than that of Host, due to the device’s lower computing power. It’s worth noting that the computing power of our baseline SSD is 42.1% slower than that of Host. Secondly, ISP models can significantly minimize the data that Host would need to retrieve over PCIe, resulting in a 50.3% average reduction in Storage. Finally, as the workloads necessitate accessing data over a filesystem, System also becomes a significant factor in Host (accounting for an average of 59.6% of total latency). When these three

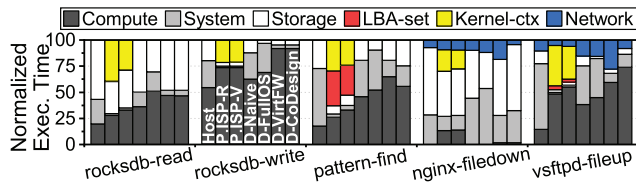


Fig. 13: Latency breakdown analysis.

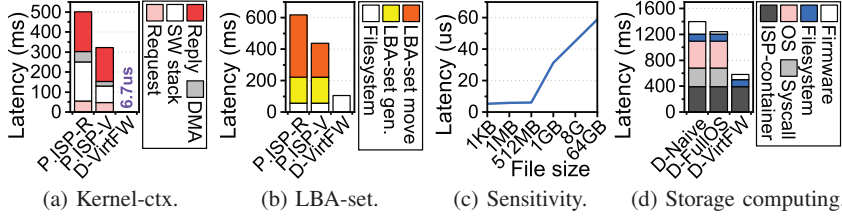


Fig. 14: Latency reduction analysis (pattern-find).

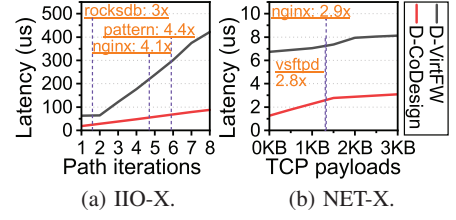


Fig. 15: HW acceleration impact.

components are the main contributors to application execution latency (and Compute is not overly heavy), $P.ISP-R/V$ can outperform *Host*; this is evident in the *rocksdb-read/nginx-filedown* workloads. For instance, *nginx-filedown* uses less processing power than any other tested workloads. Moreover, its Gets read large amounts of data for given file requests, but the number of files requested is relatively small as each file is large. This results in the shortest Kernel-ctx/LBA-set across all tested workloads. On the flip side, $P.ISP-R/V$'s System and Storage are respectively reduced by 87.3% and 50.3%, resulting in an average 26% reduction compared to *Host*.

Kernel context switches. However, in most cases for our workloads, Kernel-ctx for programmable-ISP models is longer than the reduction in data movement. On average, Kernel-ctx accounts for 27.5% of the end-to-end latency for $P.ISP-R/V$. When the target application requires system calls, it often necessitates Kernel-ctx, since $P.ISP-R/V$ lacks an OS, thereby lowering $P.ISP-R/V$'s performance. To illustrate, we present a detailed comparison of the total Kernel-ctx for $P.ISP-R/V$ and $D-VirtFW$ using *pattern-find*, with the results shown in Figure 14a. Kernel-ctx comprises the time to serialize a target word for searching in storage (*Request*) and the line number/texts containing the target word as a result (*Reply*). It also includes the time needed for users to deliver kernel contexts to the storage interface (*SW stack*) and to transfer them through PCIe (*DMA*). In contrast, $D-VirtFW$ only demonstrates Ether-oN and TCP/IP for the ISP-container offloading, reducing latency by 1.9 $\times$ and 1.4 $\times$ respectively compared to $P.ISP-R/V$. Note that, once the host delegates ISP to its SSD, $D-VirtFW$ need not engage in this kernel context delivery throughout the application execution, courtesy of containerization. The results are only returned to the host if necessary (via `docker logs`), resulting in $D-VirtFW$'s Kernel-ctx being 61.3K times shorter than $P.ISP-R/V$ ones.

Sharing of LBAs. LBA-set also becomes crucial when there are numerous file accesses; *pattern-find* in Figure 13 is a case in point (33.3% and 28.7% of $P.ISP-R/V$ latency, respectively). In practice, LBA-set consists of i) the time to generate LBA-set (by communicating with the filesystem) and ii) the time to transfer data from the host to the SSD. As demonstrated in Figure 14b, the generation and movement overheads of LBA-set contribute 32.3% and 56.9% of LBA-set on average, respectively. On the other hand, $D-VirtFW$ only displays a latency for λFS that is longer than the host filesystem of $P.ISP-R/V$ since all file accesses are directly carried out by ISP-containers (via λFS). Note that, while

LBA-set generation may seem less crucial in this analysis, its latency varies depending on application behavior. Figure 14c depicts the generation latency with varying file sizes, from 1KB to 64GB. It is important to note that it dramatically increases when the file size exceeds 512MB. This is because the *inode*'s inline data (extent-tree [77]) can only point to 512MB blocks. If the size exceeds 512MB, it necessitates accessing another block to obtain additional extent-tree(s), resulting in multiple I/O accesses for the LBA-set generation.

Implications of virtualization firmware. The process of ISP, comprised of System and Compute components, introduces significant overhead to all ISP models due to low-power computing constraints. Compute cannot be reduced by firmware (as it necessitates hardware acceleration), but as shown in Figure 13, Virtual-FW can diminish System, which comprises 31.6% of OS-level virtualization ($D-Naive/FullOS$).

We also break down Compute of $D-Naive$, $FullOS$, and $VirtFW$ into distinct latencies: "pure" firmware, filesystem, OS, system call, and ISP-container latencies. As shown in Figure 14d, $D-FullOS$ reduces the firmware latency for data movement between two disparate computing/controller complexes by 79.6% compared to $D-Naive$.

Nonetheless, both these approaches encounter inefficiencies arising from poor software management tasked with handling the OS stack and system calls. Even though $D-VirtFW$ augments the firmware latency by 2.1 $\times$ relative to $D-FullOS$, it eradicates these inefficiencies through integration of essential OS features and functional-like system call emulation. Consequently, it lowers the in-storage computing latency by 55.9% compared to $D-Naive/FullOS$.

C. Hardware Acceleration and Power/Energy Efficiencies

Sensitivity of hardware accelerations. Given the constraints of low computing power, IIO-X and NET-X play a role in curtailing ISP-container latencies. Figures 15a and 15b contrast $D-VirtFW$ with $D-CoDesign$ across varying numbers of path walking iterations and TCP/IP service payloads. In these sensitivity tests, we highlight the representative workload names where applicable.

As the number of path walking iterations escalates, there is a corresponding rise in $D-VirtFW$'s ISP-container latency. Conversely, $D-CoDesign$, assisted by IIO-X, demonstrates near-constant performance even as path walking iterations increase. The average iteration numbers of our workloads vary from 1.6 to 5.9, which augments file services by 3 $\times$ to 4.4 $\times$.

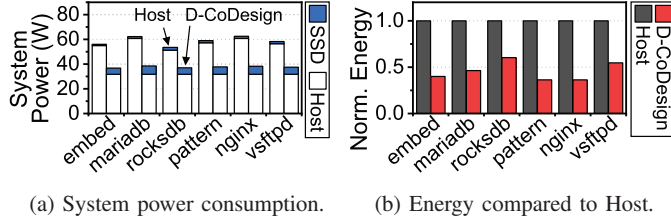


Fig. 16: System power and energy consumption.

Furthermore, NET-X expedites the end-to-end network path from TCP session management and Ether-oN request composition. As depicted in Figure 15b, D-CoDesign's NET-X keeps TCP payload latency below $3\mu s$, whereas D-VirtFW services range from 6 to $8\mu s$. However, NET-X's payload latency does increase as the payload size expands due to the need to transfer actual data via our coprocessor, which is bandwidth-limited, thus increasing NET-X cycles. Meanwhile, D-VirtFW primarily processes everything in memory and only handles the TCP/IP header, regardless of the payload size.

Power and energy efficiencies. Figures 16a and 16b scrutinize the total power and energy at the system level, including both host and SSD. Since D-CoDesign frees the system from application execution, CPU and hardware resources transition to idle, consuming 44% less power than Host. D-CoDesign consumes 3.4 times more power compared to its baseline device for ISP-containers. However, due to embedded system hardware and hardware/software optimization, its total power is 35.7% lower than Host. As analyzed in §VII-A, D-CoDesign shows reduced latency on all tested workloads. Given its low power and minimal latency, D-CoDesign exhibits superior energy behavior, 54.3% lower on average.

D. Detailed Analysis of ISP-Container

ISP-container's pure I/O performance. Figure 17a presents the average I/O latency values when both the host and ISP-container access the target storage using the workloads evaluated in §VII-B. To provide clarity, the latency of Host is normalized to D-CoDesign. On average, the ISP-container showcases a latency that is $4.1\times$ superior to that of Host. As depicted in Figure 14, the host needs to conduct data transfers via PCIe for I/O services as well as for other data exchanges, such as kernel parameters. Conversely, the ISP-container is capable of managing all these operations using internal DRAM, negating the need for mode switches, as elaborated in §V-B), thereby better performance.

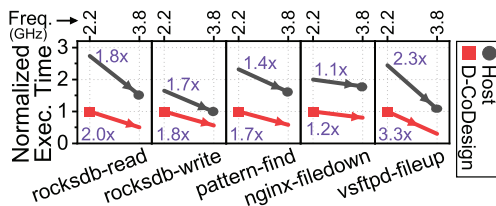


Fig. 18: Performance scaling by CPU frequency.

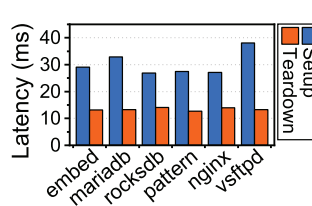


Fig. 19: Container setup/teardown.

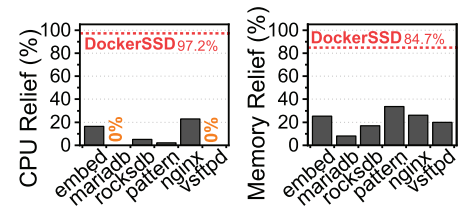


Fig. 20: Host Resource availability.

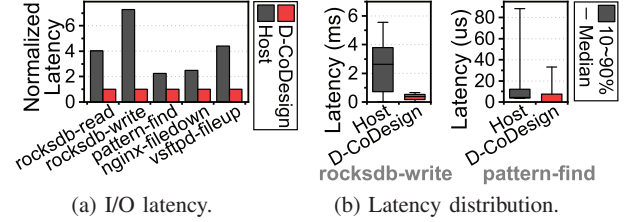


Fig. 17: I/O performance and variance.

I/O latency variation. Although D-CoDesign offers superior average latency compared to Host in terms of I/O processing, the previous evaluation may fall short in assuring consistent and sustained performance. Figure 17b displays the I/O latency distribution for the rocksdb-write and pattern-find, representing write-intensive and read-intensive tasks. This figure encompasses both the minimum and maximum latencies of the ISP-container, with the boxes indicating the latency range from the 10% to the 90%. From this illustration, it is evident that the latency distribution of D-CoDesign is both superior and more consistent compared to Host. Such a distinction arises because the Host is frequently influenced by OS-induced disruptions like interrupts and context switches. In contrast, the ISP-container operating in DockerSSD remains fully autonomous from such disturbances, hence manifesting more favorable latency distribution characteristics.

Assessing the impact of computing power. Figure 18 shows the performance trajectory of both Host and D-CoDesign when varying the CPU frequency between 2.2GHz and 3.8GHz. As the frequency of its processor escalates, D-CoDesign displays a marked enhancement in performance. This improvement can be attributed to Virtual-FW's capability to eliminate superfluous OS features and to bolster the ISP-container via hardware. Given Compute in D-CoDesign dominates execution time more than Host (cf. Figure 13), application performance is more CPU-frequency sensitive.

Analyzing container setup and teardown overhead. Figure 19 examines the time allocation for initiating and terminating the container. On average, the setup and teardown operations require 30ms and 13ms, accounting for 0.3% and 0.1% of the total container execution time (ranging from 2s to 25s), respectively. Note that setup duration expands in correlation with the number of layers comprising the target container image, while the teardown latency remains consistent across different cases. For instance, vsftpd is composed of 38 layers, while rocksdb consists of 9 layers.

Releasing host-side hardware resource. Figure 20 contrasts the availability of CPU and memory on the host side, depending on whether the target system employs DockerSSD. The resource availability exhibits fluctuations across various workloads. Without DockerSSD, the available CPU and memory resources range from 0% to 23% and 8% to 37%, respectively. However, when applications are fully offloaded as containers via DockerSSD, there is a notable increase in resource availability: 97.2% of the CPU and 84.7% of the memory become free for alternative services on the host.

VIII. RELATED WORK

ISP model analysis. We can categorize diverse ISP models into two primary groups: i) application-specific acceleration ISP models (*AppSpecific-ISP*) and ii) programmable task acceleration ISP models (*Programmable-ISP*). AppSpecific-ISP allows users to offload specific application components to the SSD using static kernels, established during device design. Examples include data filtering applications in an SSD [10, 11, 45], and endeavors to speed up key-value stack or graph processing in storage [14, 40, 41, 78]. While more advanced AppSpecific-ISP models exist [41, 44] that employ a data flow graph instead of static kernel execution [79], they are application-specific. On the other hand, Programmable-ISP offers some programmability to accommodate diverse application needs. Kernels can be crafted using APIs supplied by the device vendor [8–10, 46, 47]; it utilizes RPC or vendor commands for flexible task offloading (cf. §VII). However, users must still manually segment and adapt code for ISP.

Regrettably, most ISP models are unaware of file layouts; while a few explicitly address the LBA set [8, 46], we posit that all ISP models mentioned necessitate using the LBA set for data processing in storage. Of note, [41] does not require the LBA set exchange, but its use of a custom filesystem solely for graph processing inhibits using the SSD for conventional file storage. Similarly, kernel context switches are typically essential since existing ISPs cannot offload entire applications. Some AppSpecific-ISP models [11, 41, 44], which can directly access the underlying storage via their custom interface, do not require context switches, but these are only applicable to specific applications like graph processing.

Finally, ISP models present vulnerabilities concerning data protection. ISPs that employ custom data management interfaces [40, 41, 44] or explicit LBA set exchange [8, 46] may secure some parts of storage, but they do not allow parallel accesses on the same address range of the underlying storage. We provide a summary of the characteristics of these conventional ISP models in Table III.

Programmable hardware architecture. An alternative approach to near-flash data processing is the Computational SSD (CSD), exemplified by SmartSSDs [80, 81]. Unlike traditional ISP models, CSD introduces hardware programmability by incorporating an FPGA environment alongside existing SSDs. For instance, SmartSSDs link a Xilinx FPGA and an M.2 SSD via a soft PCIe switch. SmartSSD 1.0 [80] enables users to

	AppSpecific-ISP							Programmable-ISP			
	S-Sort [11]	Summ. [10]	KAML [40]	Graf. [41]	C.SSD [42]	D.Store [43]	H.GNN [44]	Active. [47]	Willow [8]	Biscuit [9]	Insider [46]
Dynamic kernel	×	○	×	○	×	×	○	○	○	○	○
Manual partitioning	×	×	×	×	△	×	△	×	×	×	×
File layout	×	×	×	○	×	×	×	×	○	×	○
Kernel context switches	○	×	×	×	×	×	×	×	×	×	×
Device dependency	×	×	×	×	×	×	×	×	×	×	×
Data vulnerability	×	×	○	△	×	×	○	×	△	×	○
Hardware acceleration	×	×	×	○	○	○	○	×	×	×	○

TABLE III: Comparison with conventional ISP models.

configure a range of hardware logic. However, the operating frequency of the FPGA is limited, ranging from 10MHz to 300MHz, which constrains its applicability for diverse near-storage computations. To mitigate this, SmartSSD 2.0 [81] integrates a 1.4GHz ARM hard core IP into the FPGA (Versal [82]) and connects it with the M.2 SSD.

Despite the advantage of dynamic hardware configurability, several areas for improvement remain. First, current CSD architectures lack mechanisms to synchronize with the host's operations during near-storage data processing, inheriting challenges similar to those faced by traditional ISPs. Second, CSD architectures necessitate data transfers between storage and FPGA over PCIe, since the FPGA is external to the SSD, akin to D-Naive. Third, the available FPGA logic resources are insufficient to support comprehensive computational tasks, and the costs of hardware reconfiguration, such as synthesis latency and engineering effort, are prohibitively high. Lastly, the inclusion of a soft switch within the FPGA further reduces the available logic resources.

ACKNOWLEDGEMENTS

The authors thank anonymous reviewers of ASPLOS'23, MICRO'22, OSDI'22, HPCA'22, MICRO'21, ATC'21, and FAST'21 for their constructive feedback. The work is supported in part by NRF's 2021R1A2C4001773, IITP's 2021-0-00524 & 2022-0-00117 & RS-2023-00221040 and KAIST IDEC. Donghyun Gouk and Miryeong Kwon equally contribute to this work. This work is supported by Pannesia and protected by one or more patents. Myoungsoo Jung is the corresponding author (mj@camelab.org)

IX. CONCLUSION

We propose DockerSSD, a fully flexible in-storage processing (ISP) model that can run a variety of applications near flash without source-level modification. It enables lightweight OS-level virtualization in modern SSDs, allowing the storage intelligence to harmonize with existing computing environment and makes ISP even faster. DockerSSD exhibits 2.0× faster than state-of-the-art ISP models while 1.6× and 2.3× lower power and energy, respectively.

REFERENCES

- [1] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Mallevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. Deep learning recommendation model for personalization and recommendation systems. *CoRR*, abs/1906.00091, 2019.
- [2] MariaDB. <https://mariadb.org/>.
- [3] RocksDB. <https://rocksdb.org/>.
- [4] GNU grep. <https://www.gnu.org/software/grep/>.
- [5] GNU coreutils. <https://www.gnu.org/software/coreutils/>.
- [6] Nginx. <https://www.nginx.com/>.
- [7] vsftpd. <https://security.appspot.com/vsftpd.html>.
- [8] Sudharsan Seshadri, Mark Gahagan, Sundaram Bhaskaran, Trevor Bunker, Arup De, Yanqin Jin, Yang Liu, and Steven Swanson. Willow: A User-Programmable SSD. In *The Symposium on Operating Systems Design and Implementation (OSDI)*, 2014.
- [9] Boncheol Gu, Andre S. Yoon, Duck-Ho Bae, Insoon Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanho Yoon, Sangyeun Cho, Jaeheon Jeong, and Duckhyun Chang. Biscuit: A framework for Near-data Processing of Big Data Workloads. In *The International Symposium on Computer Architecture (ISCA)*, 2016.
- [10] Gunjae Koo, Kiran Kumar Matam, I Te, HV Krishna Giri Narra, Jing Li, Hung-Wei Tseng, Steven Swanson, and Murali Annavaram. Summarizer: Trading Communication with Computing Near Storage. In *The International Symposium on Microarchitecture (MICRO)*, 2017.
- [11] Luis Cavazos Quero, Young-Sik Lee, and Jin-Soo Kim. Self-sorting SSD: Producing Sorted Data Inside Active SSDs. In *The International Conference on Mass Storage Systems and Technologies (MSST)*, 2015.
- [12] Jon Haswell. SSD Architectures to Ensure Security and Performance. *Flash Memory Summit*, 2:3, 2016.
- [13] Argonboards. LS2088A Intelligent-SSD Card. <https://www.argonboards.com/ls2088a-intelligent-ssd-card>.
- [14] Samsung. Samsung Key Value SSD enables High Performance Scaling. https://www.samsung.com/semiconductor/global.semi.static/Samsung_Key_Value_SSD_enables_High_Performance_Scaling-0.pdf, 2017.
- [15] Dirk Merkel. Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux journal*, 2014(239):2, 2014.
- [16] Myoungsoo Jung. OpenExpress: Fully Hardware Automated Open Research Framework for Future Fast NVMe Devices. In *The USENIX Annual Technical Conference (USENIX ATC 20)*, pages 649–656, 2020.
- [17] Krste Asanovic, Rimantas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelevitz, et al. The Rocket Chip Generator. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17*, 2016.
- [18] Donghyun Gouk, Miryeong Kwon, Jie Zhang, Sungjoon Koh, Wonil Choi, Nam Sung Kim, Mahmut Kandemir, and Myoungsoo Jung. Amber: Enabling Precise Full-System Simulation with Detailed Modeling of All SSD Resources. In *The International Symposium on Microarchitecture (MICRO)*, 2018.
- [19] Scott Shadley. Deployment of in-storage compute. *Storage Developer Conference*, 2018.
- [20] Layerscape 2088A and 2048A Multicore Communications Processors. <https://www.nxp.com/products/processors-and-microcontrollers/arm-processors/layerscape-multicore-processors/layerscape-2088a-and-2048a-multicore-communications-processors:LS2088A>.
- [21] Microsemi PM8609 NVMe2032 Controller. <https://www.microsemi.com/product-directory/flash-controllers/4083-pm8609-nvme2032>.
- [22] NVM Express. https://nvmexpress.org/wp-content/uploads/NVM-Express-1_4a-2020.03.09-Ratified.pdf.
- [23] Jie Zhang, Miryeong Kwon, Michael Swift, and Myoungsoo Jung. Scalable Parallel Flash Firmware for Many-core Architectures. In *The Conference on File and Storage Technologies (FAST)*, 2020.
- [24] Yong Ho Song, Sanghyuk Jung, Sang-Won Lee, and Jin-Soo Kim. Cosmos OpenSSD: A PCIe-based Open Source SSD Platform. *Flash Memory Summit*, 2014.
- [25] Qingsong Wei, Bozhao Gong, Suraj Pathak, Bharadwaj Veeravalli, LingFang Zeng, and Kanzo Okada. WAFTL: A Workload Adaptive Flash Translation Layer with Data Partition. In *The Symposium on Mass Storage Systems and Technologies (MSST)*, pages 1–12. IEEE, 2011.
- [26] Jeong-Uk Kang, Heeseung Jo, Jin-Soo Kim, and Joonwon Lee. A Superblock-based Flash Translation Layer for NAND Flash Memory. In *The International Conference on Embedded Software (EMSOFT)*, 2006.
- [27] Myoungsoo Jung, Ramya Prabhakar, and Mahmut Taylan Kandemir. Taking Garbage Collection Overheads Off the Critical Path in SSDs. In *International Conference on Distributed Systems Platforms and Open Distributed Processing (USENIX Middleware)*, 2012.
- [28] Muthukumar Murugan and David HC Du. Rejuvenator: A Static Wear Leveling Algorithm for NAND Flash Memory with Minimized Overhead. In *The International Conference on Mass Storage Systems and Technologies (MSST)*, 2011.
- [29] Wonil Choi, Mohammad Arjomand, Myoungsoo Jung, and Mahmut Kandemir. Exploiting Data Longevity for Enhancing the Lifetime of Flash-based Storage Class Memory. In *The International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS)*, 2017.

- [30] Linux Control Groups v1. <https://www.kernel.org/doc/Documentation/cgroup-v1/cgroups.txt>.
- [31] Linux Namespaces. <https://man7.org/linux/man-pages/man7/namespaces.7.html>.
- [32] Linux containers. <https://linuxcontainers.org/>.
- [33] Wang Kangjin, Yang Yong, Li Ying, Luo Hanmei, and Ma Lin. Fid: A Faster Image Distribution System for Docker Platform. In *The International Workshops on Foundations and Applications of Self* Systems (FAS*W)*, 2017.
- [34] Open Container Initiative. <https://opencontainers.org/>.
- [35] OCI runtime specification. <https://github.com/opencontainers/runtime-spec>.
- [36] OCI image specification. <https://github.com/opencontainers/image-spec>.
- [37] Docker Registry HTTP API v2. <https://docs.docker.com/registry/spec/api/#pulling-an-image>.
- [38] Docker Volumes Documentation. <https://docs.docker.com/storage/volumes/>.
- [39] Docker Engine API. <https://docs.docker.com/engine/api/latest/>.
- [40] Yanqin Jin, Hung-Wei Tseng, Yannis Papakonstantinou, and Steven Swanson. KAML: A Flexible, High-performance Key-value SSD. In *The International Symposium on High Performance Computer Architecture (HPCA)*, 2017.
- [41] Sang-Woo Jun, Andy Wright, Sizhuo Zhang, Shuotao Xu, and Arvind. GraFBoost: Using Accelerated Flash Storage for External Graph Analytics. In *The International Symposium on Computer Architecture (ISCA)*, 2018.
- [42] Shengwen Liang, Ying Wang, Youyou Lu, Zhe Yang, Huawei Li, and Xiaowei Li. Cognitive SSD: A Deep Learning Engine for In-Storage Data Retrieval. In *The USENIX Annual Technical Conference (USENIX ATC)*, 2019.
- [43] Vikram Sharma Mailthody, Zaid Qureshi, Weixin Liang, Ziyang Feng, Simon Garcia De Gonzalo, Youjie Li, Hubertus Franke, Jinjun Xiong, Jian Huang, and Wen-mei Hwu. DeepStore: In-Storage Acceleration for Intelligent Queries. In *The International Symposium on Microarchitecture (MICRO)*, 2019.
- [44] Miryeong Kwon, Donghyun Gouk, Sangwon Lee, and Myoungsoo Jung. Hardware/Software Co-Programmable framework for computational SSDs to accelerate deep learning service on Large-Scale graphs. In *USENIX Conference on File and Storage Technologies (FAST)*, pages 147–164, Santa Clara, CA, February 2022. USENIX Association.
- [45] Young-Sik Lee, Luis Cavazos Quero, Youngjae Lee, Jin-Soo Kim, and Seungryoul Maeng. Accelerating External Sorting via On-the-fly Data Merge in Active SSDs. In *The Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2014.
- [46] Zhenyuan Ruan, Tong He, and Jason Cong. INSIDER: Designing In-Storage Computing System for Emerging High-Performance Drive. In *The USENIX Annual Technical Conference (USENIX ATC)*, 2019.
- [47] Anurag Acharya, Mustafa Uysal, and Joel Saltz. Active Disks: Programming Model, Algorithms and Evaluation. In *The International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 1998.
- [48] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. Tensor-dimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 740–753, 2019.
- [49] Mark Wilkening, Udit Gupta, Samuel Hsia, Caroline Trippel, Carole-Jean Wu, David Brooks, and Gu-Yeon Wei. Recssd: near data processing for solid state drive based recommendation inference. In *The International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 717–729, 2021.
- [50] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S Lee, et al. Recnmp: Accelerating personalized recommendation with near-memory processing. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 790–803. IEEE, 2020.
- [51] Xuan Sun, Hu Wan, Qiao Li, Chia-Lin Yang, Tei-Wei Kuo, and Chun Jason Xue. Rm-ssd: In-storage computing for large-scale recommendation inference. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 1056–1070. IEEE, 2022.
- [52] Liu Ke, Xuan Zhang, Jinin So, Jong-Geon Lee, Shin-Haeng Kang, Sukhan Lee, Songyi Han, YeonGon Cho, Jin Hyun Kim, Yongsuk Kwon, et al. Near-memory processing in action: Accelerating personalized recommendation with axdimm. *IEEE Micro*, 42(1):116–127, 2021.
- [53] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. Evolution of development priorities in key-value stores serving large-scale applications: The RocksDB experience. In *USENIX Conference on File and Storage Technologies (FAST)*, pages 33–49, 2021.
- [54] Stan Hanks, Tony Li, Dino Farinacci, and Paul Traina. Generic routing encapsulation over ipv4 networks. Technical report, 1994.
- [55] iperf3. <https://iperf.fr/>.
- [56] Ioannis Sourdis, Dionisios N Pnevmatikatos, and Stamatios Vassiliadis. Scalable multigigabit pattern matching for packet inspection. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 16(2):156–166, 2008.
- [57] Xilinx. Xilinx Virtex UltraScale+. <https://www.xilinx.com/products/silicon-devices/fpga/virtex-ultrascale-plus.html>.

- [58] ARM. AMBA AXI and ACE Protocol Specification. <https://developer.arm.com/documentation/ih0022/e/>.
- [59] SiFive. SiFive TileLink Specification. <https://www.sifive.com/documentation/tilelink/tilelink-spec/>.
- [60] Myoungsoo Jung, Jie Zhang, Ahmed Abulila, Miryeong Kwon, Narges Shahidi, John Shalf, Nam Sung Kim, and Mahmut Kandemir. SimpleSSD: Modeling Solid State Drives for Holistic System Simulation. *IEEE Computer Architecture Letters*, 17(1):37–41, 2017.
- [61] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, 2011.
- [62] Sheng Li, Jung Ho Ahn, Richard D Strong, Jay B Brockman, Dean M Tullsen, and Norman P Jouppi. Mcpat: an integrated power, area, and timing modeling framework for multicore and manycore architectures. In *The International Symposium on Microarchitecture (MICRO)*, pages 469–480, 2009.
- [63] Intel SSD 750 Series. <https://ark.intel.com/content/www/us/en/ark/products/86740/intel-ssd-750-series-400gb-1-2-height-pcie-3-0-20nm-mlc.html>.
- [64] Ivan Luiz Picoli, Carla Villegas Pasco, Björn Þór Jónsson, Luc Bouganim, and Philippe Bonnet. uFLIP-OC: Understanding Flash I/O Patterns on Open-channel Solid-state Drives. In *The Asia-Pacific Workshop on Systems*, pages 1–7, 2017.
- [65] Google. gRPC. <https://grpc.io/>.
- [66] Kamyar Souri, Joao Alcantara, and Ricardo Cassia. System and method for executing native client code in a storage device, U.S. Patent 10 168 997, Jan. 2019.
- [67] Hermes Costa and Vladimir Alves. System and method for computational storage device intercommunication, U.S. Patent 10 915 381, Feb. 2021.
- [68] Mahdi Torabzadehkashi, Siavash Rezaei, Ali Heydarigorji, Hosein Bobarshad, Vladimir Alves, and Nader Bagherzadeh. Catalina: In-storage Processing Acceleration for Scalable Big Data Analytics. In *International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 2019.
- [69] Criteo Kaggle Display Advertising Challenge Dataset. <https://ailab.criteo.com/ressources/>.
- [70] TPC. TPC-H. <https://www.tpc.org/tpch/>.
- [71] Facebook. RocksDB Benchmarking Tool. <https://github.com/facebook/rocksdb/wiki/Benchmarking-tools>.
- [72] Wikipedia Dumps. <https://dumps.wikimedia.org/>.
- [73] Wayback Machine. <https://archive.org/web/>.
- [74] ImageNet 2012. <https://www.image-net.org/>.
- [75] YouTube User Generated Content Dataset. <https://media.withyoutube.com/ugc-dataset>.
- [76] Yejin Lee, Seong Hoon Seo, Hyunji Choi, Hyoung Uk Sul, Soosung Kim, Jae W Lee, and Tae Jun Ham. MERCI: efficient embedding reduction on commodity hardware via sub-query memoization. In *The International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 302–313, 2021.
- [77] EXT4 Data Layout. https://ext4.wiki.kernel.org/index.php/Ext4_Disk_Layout.
- [78] Biplob Debnath, Sudipta Sengupta, and Jin Li. FlashStore: High Throughput Persistent Key-value Store. *The VLDB Endowment*, 3(1-2):1414–1425, 2010.
- [79] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. TensorFlow: A System for Large-scale Machine Learning. In *The Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.
- [80] Xilinx Samsung. Samsung SmartSSD 1.0. <https://download.semiconductor.samsung.com/resources/brochure/Samsung%20SmartSSD%20Computational%20Storage%20Drive.pdf>, 2021.
- [81] Xilinx Samsung. Samsung SmartSSD 2.0. <https://samsungmsl.com/smartssd2/>, 2022.
- [82] Xilinx. Xilinx Versal Adaptive SoC. <https://www.xilinx.com/products/silicon-devices/acap/versal.html>.